TMax:
A simple recipe for terminal agents

Hamish Ivison[*,α,ω] **Junjie Oscar Yin**[*,ω]
Rulin Shao[α,ω] **Teng Xiao**[α] **Nathan Lambert**[α] **Hannaneh Hajishirzi**[ω]
[α]Allen Institute for AI [ω]University of Washington
{hamishiv,osey}@cs.washington.edu

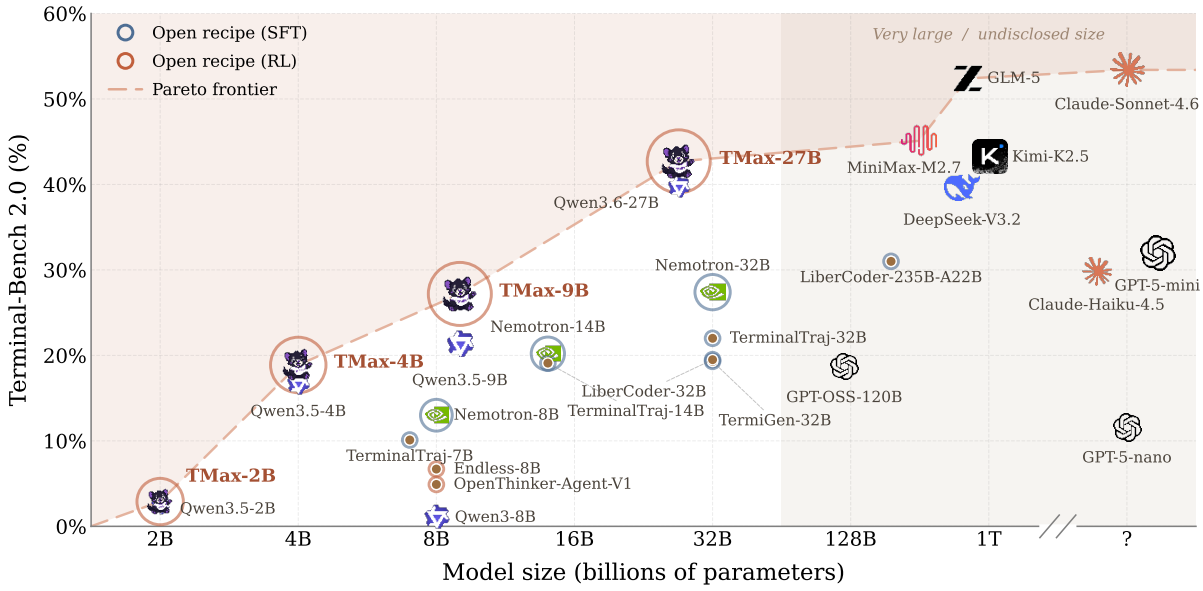

Figure 1: Performance of TMax models compared to prior work with open data and selected closed and open-weight models on Terminal-Bench 2.0. For TMax and Qwen models, we report scores using our simple harness. **TMax outperforms prior work with open data (especially prior open RL recipes) and dominates the Pareto curve for models under 32B parameters**. For further details see §E.1.

Abstract

Terminal-using agents have quickly become the most popular downstream application of language models (LMs). Despite their prevalence, relatively little academic work has examined RL-based training of these models, likely due to difficult benchmarks, a lack of data, and a lack of simple baseline recipes. We present TMax, the strongest open RL recipe for terminal agents to date, bringing open data recipes closer to the frontier. While simple, our recipe achieves **27% on Terminal-Bench 2.0 with only 9B parameters**, outperforming much larger models from prior work. Concretely, we generate data using a novel taxonomy, combining difficulty control, personas, and verifier diversification, which allows us to cheaply generate large amounts of terminal environments for RL and SFT training. We open-source our terminal dataset, which is over 2.5x larger than previously released terminal-agent datasets. We then train open-weight models using RL with our data, using a simple, outcome-only recipe. We release our data, models, and code as a strong baseline for future open academic work on terminal agents at https://github.com/hamishivi/tmax.

1 Introduction

Terminal-based agentic coding products have quickly become incredibly popular, with models expected to perform increasingly complex and long-running tasks through the terminal (Anthropic, 2025; Cursor Research et al., 2026). However, existing academic work has largely focused on bug-fixing setups (Jimenez et al., 2024) or relatively simple terminal tasks (Team, 2025; Gandhi et al., 2025) as opposed to complex, long-horizon terminal tasks as exemplified by Terminal-Bench (Merrill et al., 2026). In this work, we aim to close this gap, providing both a large dataset of complex terminal tasks for training and a recipe for training small yet powerful terminal agents using

[*]Equal contribution. Work done while HI and NL were at Ai2.

open-weight models, serving as a base for future research on terminal agents. Our data generation and model training recipes are simple yet effective, serving both as strong baselines and strong models in their own right.

First, we introduce a new dataset, TMAX-15K, comprised of 14,600 reinforcement learning (RL) environment instances varying in their difficulty, domain, and skills required. We generate our data through a powerful yet simple synthetic data pipeline, relying on a strong frontier model to generate useful environments for us. As opposed to prior work, we explicitly control and increase the difficulty of our tasks, and go beyond simple binary correctness checks to continuously-valued rewards for certain tasks. **TMAX-15K is over 2.5x larger than the second largest terminal dataset (that releases full environment data), and is significantly harder than most prior work** as judged by Gemini-3-Flash-Preview pass rates.

Given this data, we then explore how to perform post-training of terminal agents with a focus on RL training. Despite multiple works on generating terminal agent data (Wu et al., 2026; Zhu et al., 2026), relatively little work has explored the RL training side. We develop and train models in a fully asynchronous RL infrastructure based on open-instruct (Lambert et al., 2025). We find naive GRPO struggles to remain stable in long-horizon, agentic scenarios, and so train our models using Divergence Proximal Policy Optimization (DPPO) (Qi et al., 2026) with an FP32 LM head and a large group size to improve stability. **Our best 9B model achieves 27% on Terminal-Bench 2.0**, outperforming other open models at similar sizes and performing similarly to smaller closed-source models such as Claude Haiku 4.5. We apply our recipe to train terminal agent models on a variety of datasets, and find that using our newly generated data results in the strongest performing model.

We additionally investigate how well our training generalizes across tasks, and find that **our RL training improves other agentic evaluations such as SWE-Bench Verified (Jimenez et al., 2024) by over 5 points, as well as non-agentic evaluations such as AIME**. Finally, we also find our RL training generalizes across harnesses, improving performance even when using different prompts and toolsets to those used during RL training. All in all, this shows that our RL recipe does not simply perform 'harness-fitting', but teaches the model new skills and capabilities that generalize across settings.

We hope that our overall recipe and data prove useful for future academic work exploring terminal agents, as we believe this setting provides a number of unique and interesting challenges, including but not limited to training stability, dealing with complex tool call setups, harness improvements, and further improving performance on more difficult downstream tasks. Concretely, our contributions are:

- We introduce TMAX-15K, a dataset of 14,600 RL environment instances, over 2.5x larger than prior terminal datasets.
- We train open-weight terminal agents and achieve state-of-the-art performance among open models under 30B parameters under default Terminal-Bench settings. Our best 9B model reaches 27% on Terminal-Bench 2.0.
- We provide a simple, reproducible open RL recipe for achieving this performance and publicly release all elements required for training our models. Our recipe outperforms prior open RL recipes such as Endless Terminals and OpenThinker-Agent.
- We show that terminal-based RL training can generalize non-trivially across harnesses and tasks, providing strong evidence our training teaches the model powerful new capabilities.

We additionally publicly release code, checkpoints and data associated with this work for future use and reproduction at `https://github.com/hamishivi/tmax`. Included are training artefacts such as RL rollouts and logprobs from the training of TMAX-9B, making analysing our runs significantly easier.

2 Background and Related Work

2.1 Data for Terminal Agents

Early work on terminal agents focused on training models to translate natural language to bash statements. NL2Bash (Lin et al., 2018) paired short instructions with bash commands mined from the web, and was converted into an RL dataset for the training of OpenThinker Agent (Team, 2025). However, it failed to yield significant improvements over their strong SFT model. This showcases

the need for generating more complex terminal-agent tasks, which provide stronger learning signals on more complex tasks.

As such, recent data generation work for terminal agents attempts to synthesise entirely new tasks in two different ways: *adapting* existing repositories or tasks into terminal tasks, or *synthesising* entirely new tasks using only seed tasks and taxonomies to guide the generation.

The first category is largely dominated by datasets and models targeting SWE-Bench (Jimenez et al., 2024), including SWE-Universe (Chen et al., 2026), SWE-rebench (Badertdinov et al., 2025), SWE-Gym (Pan et al., 2024), SWE-Smith (Yang et al., 2025a) and SERA (Shen et al., 2026a). These largely base their tasks and setups off issues and pull requests in online repositories, and so focus on bug-fixing tasks that only represent a subset of tasks that terminal agents are typically expected to accomplish (e.g., environment setup, model training, and writing entirely new codebases from scratch). TerminalTraj (Wu et al., 2026) also generates data from existing repositories whilst focusing on more generic terminal tasks, but validates their approach only using SFT training, and we find in practice the diversity of their tasks is biased toward software engineering tasks (Fig. 3), likely due to their repository-based approach.

The second category generates data from a given taxonomy or set of seed tasks, and is the approach we take in this work. We are particularly inspired by Endless Terminals (Gandhi et al., 2025), which similarly utilizes a strong external model to generate new terminal tasks, but they generate significantly fewer tasks that are largely focused on file manipulation, making them too easy for modern models. For instance, we find that Gemini-3-Flash achieves over 90% pass@1 on Endless Terminals. TermiGen (Zhu et al., 2026) similarly generates from categories and seed tasks, but focuses on using them to construct good SFT data, and does not control the difficulty of their tasks. Nemotron-Terminal (Pi et al., 2026) both generates new data from seed prompts and adapts existing datasets into terminal tasks, but does not release any RL environment data, and similarly only validates their data through large-scale SFT training. SETA (Shen et al., 2026b) and LiteCoder (Peng et al., 2026) both follow a similar taxonomy-plus-seed recipe at a few hundred examples, which limits their utility as comparison points. In contrast, we release over 14,000 unique RL environments, with diverse difficulties and well-balanced across a range of terminal-agent tasks.

2.2 RL training for Terminal Agents

With the rise of reasoning models (Guo et al., 2025), reinforcement learning has become a staple in the LM post-training pipeline, including for terminal-agent training. However, relatively little work examines RL training for terminal agents, with most prior data-generation work only using their data for model finetuning (Wu et al., 2026; Zhu et al., 2026; Pi et al., 2026), despite the success of RL in training strong models for agentic tasks (Luo et al., 2025; Cursor Research et al., 2026).

Closest to our work is Endless Terminals (Gandhi et al., 2025), who perform RL training over a Qwen 3 8B model with their generated data, using a simple harness and PPO over limited context lengths (16k tokens). We instead train over longer context lengths, with a larger set of more difficult tasks using GRPO. We show in §4.2 that our dataset yields improved performance over Endless Terminals, which we attribute to our closer focus on the increased and more varied difficulty of our generated tasks. SETA (Shen et al., 2026b) and OpenThinker-Agent (Team, 2025) also perform RL training for terminal agents, but demonstrate limited improvement over their initial SFT checkpoints (in the range of 1 point), while we provide a setup in which RL training provides significant improvements (over 5 points improvement), providing a stronger signal for developing and testing improvements to agentic RL. ROME (Wang et al., 2026b) also explores RL training for terminal agents and notes a number of difficulties in their training, but does not release data or setup for replicating and developing on their setup, and develops a bespoke algorithm for training. In contrast, we find that relatively simple RL recipes work for training, and release all data, models, and code for our training.

3 Terminal Data Generation

Training autonomous terminal agents requires large-scale and diverse coding tasks. With real-world terminal data often scarce or proprietary (Merrill et al., 2026), synthetic generation is used to bridge the data gap (Pi et al., 2026; Zhu et al., 2026).

However, existing pipelines fall short along three axes. First, they rely on complex multi-stage gen-

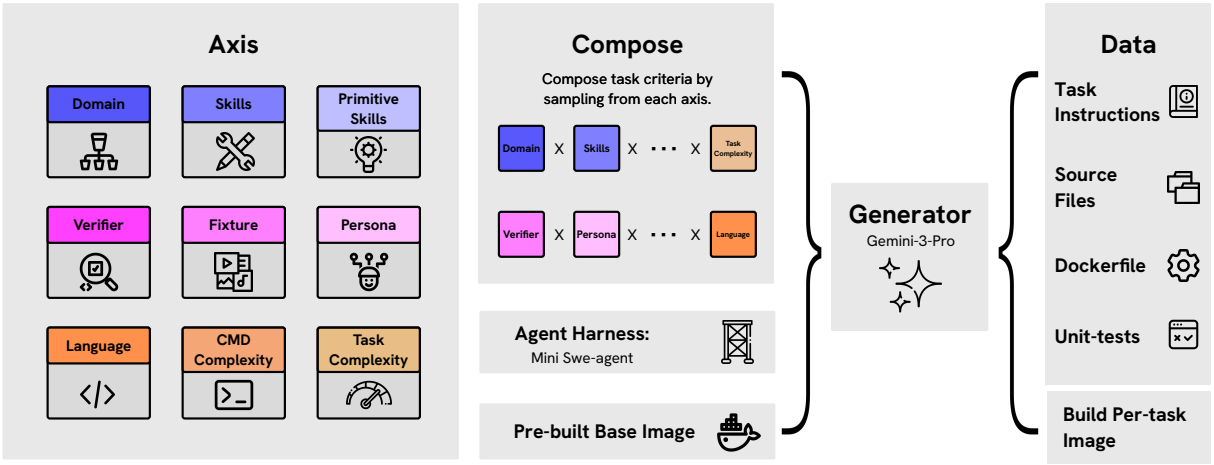

Figure 2: **TMax Data Pipeline**. Each task is composed by hierarchically sampling from 9 structured axes, after which a data generator instantiates into a Dockerfile, unit-test verifier, source files, and task instructions. Tasks are built atop a pre-built per-domain base image and served through a mini-SWE-agent harness. Composing axes yields combinatorially many task signatures with explicit, per-axis control over difficulty and diversity. The single build step, not needing teacher-based validation, keeps generation cheap to scale.

eration procedures that are hard to scale. Second, they generate homogeneous task suites with limited coverage against real-world tasks and Terminal-Bench (Merrill et al., 2026). Third, they offer little control over difficulty, often producing bi-modal task pools that are either trivially solved or unsolvable by current models.

Hence, we propose a simple terminal environment generation pipeline that is scalable, diverse, and difficulty-aware. We use Gemini-3-Pro (Pichai et al., 2025) as our generation model, motivated by its strong performance on Terminal-Bench.

3.1 Generation pipeline.

We adopt a simple compositional generation framework: each synthetic task is sampled as a product of structured axes (Table 8). We follow Pi et al. (2026) to seed the first two axes – domain and skills – and introduce six orthogonal axes targeting diversity and difficulty.

Scalability via soft filtering. Unlike prior works that validate task quality and correctness through expensive teacher generation (Zhu et al., 2026; Wu et al., 2026), we deliberately skip this validation step entirely, as our RL training applies effective soft filtering. Specifically, our RL infrastructure (§4.1) filters out samples for which the policy pass rate is 0, since these contribute no gradient. In practice, we find the all-zero rate for our generated data is generally low (< 8 samples filtered per batch, see §D.5). We only need to ensure environment executability by building a Docker image per task. We pre-configure these images per domain so tasks within the same domain share the same base image, with task-specific dependencies added as needed. This simplifies the two-step approach to one, enabling efficient, synthetic environment scaling.

Diversity via hierarchical sampling. The compositional sampler is itself our main diversity mechanism, and drawing from each axis yields combinatorially many distinct task signatures. Further, we add two specific diversity dimensions:

Persona diversification. We augment generation with a set of user personas, motivated by prior work on persona-conditioned data synthesis (Ge et al., 2025; Lambert et al., 2025), to further diversify generated tasks. Personas are domain-specific (5–18 per domain, e.g. "red-team operator crafting an evasion payload" for security), so each is always reasonable for its sampled skill set.

Multi-modal fixtures. Previous RL tasks are all *text-in / text-out* problems. Here we extend the task inputs beyond plain text by shipping a concrete artifact per task: a PNG image, audio file, video, stripped binary, vendored package, or multi-service compose stack (Table 10). The policy itself remains a text-only language model and training is unchanged: rather than perceiving these artifacts natively, the agent inspects them through standard terminal tooling (e.g., OCR, audio transcription, `ffmpeg`), so no multi-modal model is required.

Difficulty via explicit calibration. Unlike previous works, we make an explicit effort to calibrate task difficulty, avoiding the bi-modal distribution where tasks are either trivially solved or unsolvable by current models (Table 1). We address this through two mechanisms:

Fine-grained complexity. We introduce two complexity axes, *task complexity* (from a few shell commands to intricate workflows of 30–60 commands) and *command complexity* (from bash-only to bash + code + system services), for granular control over how hard each task is. We sample uniformly across complexity buckets by default, with optional per-bucket up-weights to match specific model capabilities or induce a curriculum.

Graded verifiers. Previous RL tasks rely on exact string equality against a ground-truth answer. Here we extend with richer verifier kinds (Table 9): metric-threshold (e.g., accuracy ≥ 0.95), adversarial-corpus (accept clean, reject malicious), fuzz-equivalence (bit-exact match against an oracle), or multi-protocol (protocol-level requests against a service). Thresholds give us a continuous difficulty knob, while multi-condition variants naturally extend task length.

Using this pipeline, we generate a dataset of 14,600 tasks, which we name TMAX-15K[1].

3.2 Comparing Terminal Datasets

We compare our generated data against past work, both against prior terminal agent datasets, and a prior SWE-bench-targeting dataset, SWE-Smith (Yang et al., 2025a).

Composition A good training dataset should have broad coverage and a balanced composition across target domains. By design, our generation framework exposes domain as an explicit sampling axis, so we can directly calibrate per-domain mass at generation time. To compare across datasets, we use Gemini-3-Pro to annotate every task for our and the six comparison datasets, with one of these nine domain labels. As shown in Figure 3, prior terminal datasets concentrate 34–95% of their mass on a single domain (e.g., `software_engineering` accounts for 95% of SWE-Smith and over 60% of TerminalTraj and CLI-Gym), while ours spreads roughly uniformly across all nine. This balanced coverage avoids biasing the policy toward any narrow domain and better covers real-world usage.

[1]as 14,600 rounds to roughly 15k.

Difficulty & Balance To compare difficulty across datasets, we evaluate Gemini-3-Flash-Preview on a fixed subsample of 250 tasks per dataset (8 rollouts each) and report mean pass@k in Table 1. Our data is among the most challenging overall: pass@1 is 42% (vs. 41–92% for prior datasets), and it has the lowest pass@4 (50%) and pass@8 (53%) of any dataset, indicating the difficulty gap persists as we draw more rollouts (§B.2).

We additionally compute a *balance score* for each categorical axis (domain, skill-type, task complexity, command complexity) as the fraction-of-uniform diversity of its empirical distribution:

$$\text{Balance} = \frac{\exp(H)}{N}, \qquad H = -\sum_{i=1}^{N} p_i \log p_i, \tag{1}$$

where p_i is the proportion of tasks in bucket i and N is the number of buckets. The score lies in $[1/N, 1]$ with 1 being perfectly uniform; see §B.3 for derivation. Our data attains the highest balance on both domain (0.998) and skill-type (0.732).

De-contamination We measure overlap between dataset task descriptions and the Terminal-Bench and TB-Lite tasks using a 13-gram sliding window, following standard contamination protocol (Brown et al., 2020; Touvron et al., 2023). As shown in Table 11 (§B.4), our data shows 0% overlap with both benchmarks, on par with the majority of prior datasets.

3.3 SFT Data Generation

We additionally generate a small SFT dataset to use as a warm-start for RL training, with prior works showing that warm-up could help training stability and performance (Guo et al., 2025; Team et al., 2025). Re-using our terminal data pipeline, we additionally generate 2.2k environments, and we use Qwen 3.6 27B to generate 8 trajectories for each environment. We filter out trajectories with unparsed tool calls.[2] Together, this yields 16.5K total SFT trajectories, with 8K successful trajectories. We use this data for SFT experiments and finetuning Qwen 3 8B.

3.4 Agent Harness

Unless otherwise stated, we use a simple harness based on mini-SWE-agent (Yang et al., 2024) with

[2]To avoid traces containing the Qwen 3 XML-style tool call format, which can confuse small models with different tool call formats like Qwen 3 8B.

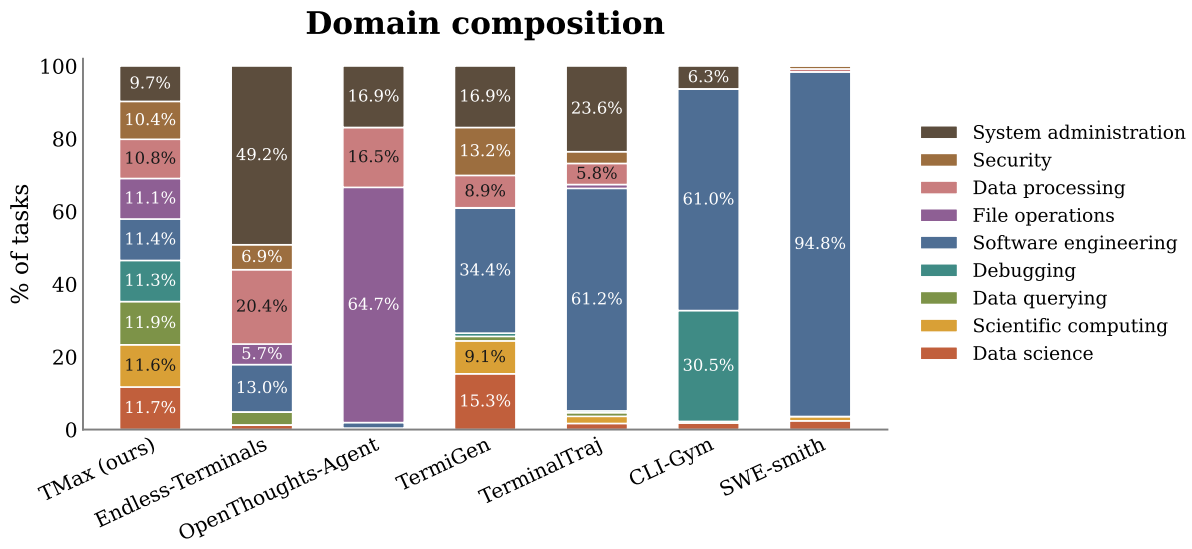

Figure 3: **Data Composition**. Domain distribution of tasks across terminal datasets. Prior datasets skew heavily toward one or two domains, whereas our compositional sampler yields balanced coverage across all nine domains.

Data	Dataset size	Pass@1 Gemini	Pass@4 Gemini	Pass@8 Gemini	Mean turns	Mean tokens / run (K)	Domain balance	Skill-type balance
TMax (Ours)	15k	42%	50%	53%	16.3	120K	0.998	0.732
Endless Terminals	2.4k	92%	94%	95%	15.7	117K	0.481	0.284
Open Thoughts Agents	0.7k	51%	58%	60%	19.3	106K	0.292	0.153
Terminal Gen	3k	57%	64%	66%	20.9	305K	0.646	0.477
Terminal Traj	5.5k	54%	63%	65%	15.1	141K	0.363	0.374
CLI-Gym	1.5k	41%	52%	55%	43.8	769K	0.283	0.061
SWE-Smith	59k	54%	69%	72%	41.2	582K	0.146	0.042

Table 1: **Terminal datasets and difficulty-related statistics** with Gemini-3-Flash-Preview, evaluated on a fixed subsample of 250 tasks per dataset with 8 rollouts each. Pass@k is mean pass@k across tasks. Mean tokens/run is the sum of tokens over turns, in thousands. Domain and skill-type balance are uniformity scores in $[0, 1]$. We note that we find that most tasks in TMAX-15K achieve reward > 0 by TMAX-9B at least once over 32 rollouts (§D.5).

persistent shell. We find that Terminus-2 harness (Merrill et al., 2026) is more brittle with small models, as it requires agents to send raw keystrokes to interact with the terminal. See §C for more details.

4 Training Terminal Agents

We now validate our data by practically applying it to RL training. We aim to validate our data with a simple recipe, providing a testbed in which we believe there is significant room for improvement.

4.1 Experimental Setup

Algorithm We train models using DPPO (Qi et al., 2026), which is a variant of GRPO (Shao et al., 2024) which masks tokens when inference and training logprobs deviate. Specifically, we mask tokens based on a binary approximation of the total variation (TV) divergence. We use a token level loss (Yu et al., 2025) and run training fully asynchronously. We filter groups with zero standard deviation and use active sampling to ensure full batches following Olmo 3 (Olmo et al., 2026).

Infrastructure We extend open-instruct (Lambert et al., 2025) for terminal agent training. As such, we use vLLM (Kwon et al., 2023) for rollouts, and use either a Podman or Apptainer backend for managing sandboxes. For rollouts, we use the same mini-SWE-agent-based harness as used for verification and SFT data generation. We additionally ensure that the language model head of models is computed and kept in FP32 precision to minimize training-inference mismatch, following MiniMax et al. (2025). We find this especially crucial when training Qwen 3.5. To highlight this, we plot the maximum logprob difference during RL training of Qwen 3.5 and Qwen 3 in Figure 4. Using the FP32

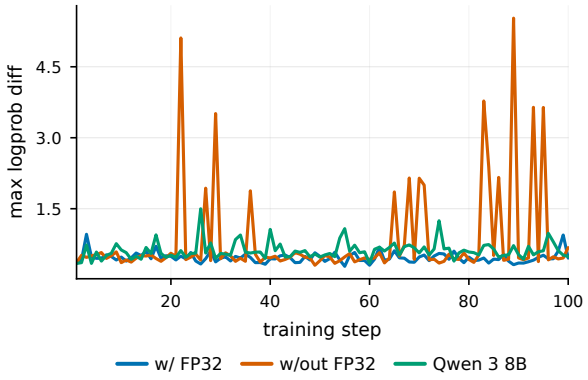

Figure 4: **Maximum difference between inference (vLLM) and trainer (HuggingFace) logprobs** during the first 100 steps of RL training for Qwen 3.5 9B. Qwen 3.5 without the FP32 LM head displays larger and more frequent spikes. Qwen 3 8B training also does not display spikes, even though we do not apply the FP32 LM head.

LM head reduces the maximum difference dramatically. Interestingly, we find this is less important for Qwen 3, which does not display large logprob differences even without the FP32 head. We train on a cluster of H100 nodes, and typically use 2 nodes for training and 6 for inference. Training takes 2–3 days depending on sequence length and infrastructure stability. We further discuss training stability in §5.2.

Evaluation We primarily evaluate on Terminal-Bench 2.1 (Merrill et al., 2026) and Terminal-Bench Lite (OpenThoughts-Agent team, 2026). We use Harbor (Harbor Framework Team, 2026) and a Podman-based backend for evaluation.

For our final evaluations as shown in Fig. 1, we use Terminal-Bench 2.0 (the earlier version) to compare directly with past work. Additionally, we use Daytona[3] as the backend as recommended by Terminal-Bench authors. We find that the choice of backend can result in varied performance: for instance, Daytona sandboxes often perform installs faster than our locally-hosted runs, resulting in fewer timeouts. However, the cost of these services makes it expensive to develop against.[4] We plan to improve our infrastructure for local rollouts in future work via more dedicated sandbox services.

As Terminal-Bench tasks have per-task timeouts, different inference setups can yield different model performances based on inference throughput. To keep settings fair, we run all models on a single A100 node with vLLM, and run each evaluation 3 times to reduce noise unless otherwise stated. We do not override timeout defaults. We reuse this setup when running other evaluations, including SWE-Bench Verified (Jimenez et al., 2024) and AIME.

Hyperparameters We run RL training for 500 steps unless otherwise stated, and choose the best checkpoint based on Terminal-Bench Lite performance, evaluated every 100 steps. We use a group size of 32 and 8 prompts per batch, with a maximum sequence length of 65536 tokens and 64 maximum tool calls. When training Qwen 3.5 and 3.6 models, we do not perform any initial SFT. For SWE-Smith, we do only 100 steps of training due to extremely high solve rates which prevent effective training (see §D.3). We edit chat templates such that thinking is preserved on intermediate turns (often called 'interleaved thinking'), which has been shown to improve performance in agentic settings (MiniMax, 2025). We show further hyperparameters in §D.1.

4.2 Core Results

TMAX-15K outperforms other terminal datasets. We first apply RL training to Qwen 3.5 9B (Qwen Team, 2026), comparing training on our data against a variety of datasets from prior work in Table 2. We include prior datasets focusing on Terminal-Bench alongside SWE-Smith, as a representative sample of a dataset focusing on SWE-Bench tasks. We note that in many cases, we are the first to openly apply these datasets for RL training (e.g., in the case of TermiGen and TerminalTraj). We find that training on TMAX-15K results in strong Terminal-Bench Lite and Terminal-Bench 2.1 performance, which we attribute to its improved difficulty and diversity. We call the model trained on our data TMAX-9B.

To further investigate this, we plot the average number of steps taken by the model during the first 300 steps of training for each dataset in Fig. 5, and find that the model uses more steps on average when training on TMAX-15K than other datasets over the course of training, and especially towards later steps. This suggests that our data remains difficult throughout training, requiring the model to perform many steps and more complex tasks.

We additionally investigate how TMAX-9B changes in thinking and tool-use patterns over the

[3] https://www.daytona.io/

[4] One RL training run of TMAX-9B would cost roughly $3,150 on Daytona, using average runtimes from the run and pricing from https://www.daytona.io/pricing.

RL Dataset	TB Lite	TB 2.1
None (i.e., Qwen 3.5 9B)	41.9 ± 2.7	16.1 ± 3.7
TermiGen (Zhu et al., 2026)	49.4 ± 1.5	25.1 ± 1.9
Endless Terminals (Gandhi et al., 2025)	52.6 ± 1.4	25.5 ± 1.4
OpenThinker-Agent (Team, 2025)	53.0 ± 0.7	25.1 ± 3.7
TerminalTraj (Wu et al., 2026)	45.8 ± 2.7	18.0 ± 0.0
CLI-Gym (Lin et al., 2026)	50.7 ± 5.9	25.1 ± 1.4
SWE-Smith (Yang et al., 2025b)	47.2 ± 2.2	21.0 ± 0.5
TMAX-15K (Ours)	$\mathbf{57.2 \pm 2.5}$	$\mathbf{28.8 \pm 1.4}$

Table 2: **Training on TMAX-15K results in strongest Terminal-Bench performance.** Performance of models after RL training on the given dataset on Terminal-Bench Lite and Terminal-Bench 2.1. We report mean and stderr over 3 runs.

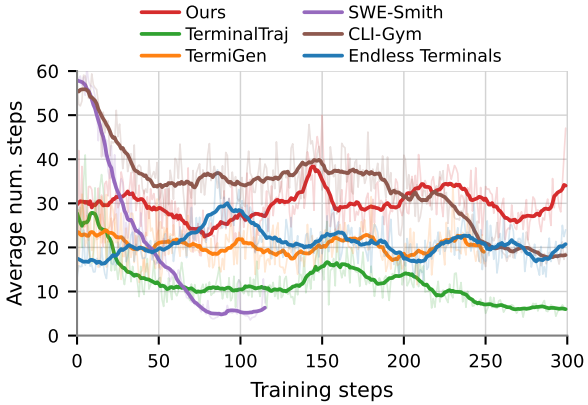

Figure 5: Average step count over RL training when training Qwen 3.5 on different datasets. We smooth using a 15-step average window. **Training on TMAX-15K consistently uses higher steps than other datasets.** SWE-Smith training is shorter than others for reasons given in §D.3.

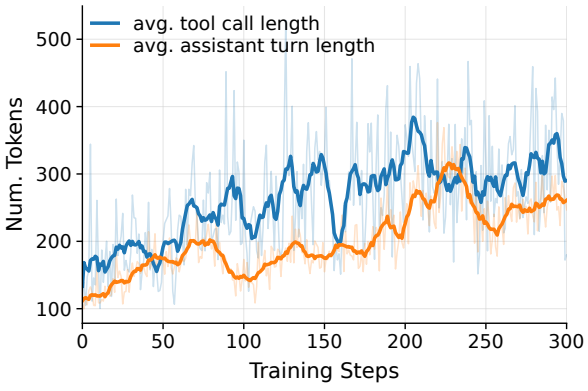

Figure 6: Average length (in tokens) of assistant turns and tool calls. We do not include tool calls in the assistant turn length. Per-turn output length gradually increases over the course of RL training, suggesting **the model learns to better make use of inference-time scaling**.

course of training. We find that the average number of tokens used during assistant turns increases (both thinking and tool-calling tokens), as shown in Fig. 6. This is reminiscent of inference-time reasoning scaling in single-turn math settings (Guo et al., 2025), and suggests that the model is gradually learning more complex reasoning and more complex tool calls over the course of training, resulting in its improved performance.

TMAX-9B outperforms prior small models on Terminal-Bench 2.0. As seen in Fig. 1, TMAX-9B is the strongest model under 10B parameters, even outperforming the 32B variants of prior work, and achieving performance close to closed offerings from large labs such as Claude Haiku 4.5. This highlights that TMAX-9B is already a strong model in its own right, beyond serving as a strong baseline approach for future work. Additionally, we outperform prior open RL recipes for terminal agents such as Endless-8B (Gandhi et al., 2025) and OpenThinker-Agent (Team, 2025), making our overall recipe the strongest so far. We attribute this to our novel generated dataset, our use of a stronger base model, and our improved RL recipe.

TMAX RL training improves models across different sizes. We additionally apply our recipe, without modification,[5] to the other sizes of Qwen 3.5, resulting in TMAX-2B, TMAX-4B and TMAX-27B. We show evaluation results in Table 3. In all cases, we improve over the Qwen 3.5 baseline, although the gap grows smaller as model size reduces, likely due to the limited capacity of smaller

[5]For Qwen 3.6 27B, we only train to 300 steps and evaluate at steps 160 and 240 additionally, due to its greater training cost.

Model	TB Lite	TB 2.1
Qwen 3.5 2B	5.7 ± 1.6	1.9 ± 1.4
TMAX-2B	$\mathbf{11.8 \pm 1.4}$	$\mathbf{4.2 \pm 1.2}$
Qwen 3.5 4B	31.8 ± 3.8	14.2 ± 2.3
TMAX-4B	$\mathbf{42.6 \pm 1.5}$	$\mathbf{19.9 \pm 1.1}$
Qwen 3.5 9B	41.9 ± 2.7	16.1 ± 3.7
TMAX-9B	$\mathbf{57.2 \pm 2.5}$	$\mathbf{28.8 \pm 1.4}$
Qwen 3.6 27B	$\mathbf{70.8 \pm 2.1}$	40.5 ± 2.4
TMAX-27B	68.6 ± 4.7	$\mathbf{44.9 \pm 1.8}$

Table 3: **We find that TMAX RL training improves over their starting point models at all sizes**, although the gap is biggest for TMAX-9B. Performance of Qwen 3.5/3.6 and TMAX models on Terminal-Bench lite and Terminal-Bench 2.1.

Model	Harness	SBV	AIME
Qwen 3.5 9B	None	–	67.5 ± 4.9
Qwen 3.5 9B	Ours	44.0 ± 2.0	73.3 ± 2.7
+ RL (TMAX)	Ours	$\mathbf{53.5 \pm 0.6}$	$\mathbf{91.1 \pm 1.6}$

Table 4: **Improved performance generalizes across tasks.** Performance of Qwen 3.5 9B on SWE-Bench Verified and AIME'24/25. We show mean and stderr over 3 evaluation runs. We evaluate AIME both in terminal-agent and single-turn settings.

models to learn complex agentic behaviours. As for TMAX-27B, we believe that its base (Qwen 3.6 27B) has undergone additional training relative to the Qwen 3.5 series aking it much harder to improve.

4.3 Generalization of TMAX-9B

We next investigate how well TMAX-9B generalises across three important axes: tasks (i.e., what it is asked to do), harnesses (i.e., what tools and prompts it is provided when performing tasks), and model families (i.e., the starting point model).

TMAX RL training generalises to other tasks. We next investigate if our RL training extends to evaluations beyond Terminal-Bench in Table 4. We evaluate TMAX-9B on SWE-Bench Verified and AIME 2024/2025, and additionally compare to AIME performance in a traditional single-turn no-harness setting. Encouragingly, we find that performance improves across the board by significant margins, suggesting that the model is not simply fitting to the harness and domain, but also learning how to better make use of its terminal tools to solve generic problems. This aligns with prior work showing that terminal-agent training can improve general model capabilities (Cheng et al., 2026).

TMAX RL training generalises to other harnesses. We then investigate if our gains are limited to our harness setup by evaluating Qwen 3.5 9B and TMAX-9B with varied harnesses. We find that TMAX-9B improves by at least 9 points in all harnesses, although its largest gains and strongest performance remain in our own. This suggests that terminal RL training on a single harness can generalise across other setups, contrary to recent work (Wang et al., 2026a).

TMAX RL training improves different model families. Finally, we also apply RL training to Qwen 3 8B to see how it transfers to a different model family. We finetune Qwen 3 8B on the small SFT dataset described in §3.3, and then apply the same recipe.[6] We find that Qwen 3 8B similarly shows strong improvements in Terminal-Bench Lite, although less so in Terminal-Bench 2.1, likely due to its harder difficulty making smaller model improvements hard to observe.

Taken together, these results strongly suggest that **TMAX RL training teaches the model new capabilities and improved terminal use**, as opposed to 'simply' learning our specific harness, overfitting on the types of tasks found in Terminal-Bench, or just being a feature of Qwen 3.5 models.

5 Challenges in training TMAX-9B

5.1 Qwen 3.5 does not benefit as much from existing SFT datasets.

Common post-training pipelines usually perform SFT training to 'warm-start' the model before RL training (Guo et al., 2025; Olmo et al., 2026). However, we find that existing datasets seem to degrade Qwen 3.5's performance, likely due to it having undergone extensive post-training already. In Table 7, we compare Qwen 3.5 9B and Qwen 3 performance before and after performing SFT on both the SFT mixture mentioned in §3.3 and a larger mixture composed from our SFT data mixed with data from past work (Pi et al., 2026; Wu et al., 2026; Team, 2025; Shen et al., 2026a).[7] We find that the larger mix significantly improves performance on

[6] See §D.2 for more SFT details. For RL, we apply the same hyperparameters, but reduce maximum sequence length to 32768 at train time and 40960 at evaluation time to reduce computational cost.

[7] See §D.2 for details.

Model	Ours	OpenHands	mini-SWE-agent	Terminus-2
Qwen 3.5 9B	41.9 ± 2.7	36.0 ± 2.8	44.1 ± 3.3	36.4 ± 2.2
TMAX-9B	$\mathbf{57.2 \pm 2.5}$	$\mathbf{46.9 \pm 3.7}$	$\mathbf{55.3 \pm 4.5}$	$\mathbf{45.3 \pm 2.4}$

Table 5: **Improved performance generalizes across harnesses.** Terminal-Bench Lite performance of Qwen 3.5 9B across evaluation harnesses. Mean $\pm$ stderr over 3 runs.

Model	TB Lite	TB 2.1
Qwen 3 8B	7.3 ± 1.0	1.1 ± 0.9
+ SFT	11.5 ± 0.1	$\mathbf{6.0 \pm 1.4}$
+ RL	$\mathbf{17.7 \pm 1.9}$	5.2 ± 2.3

Table 6: **TMAX RL also improves Qwen 3.** Performance of Qwen 3 8b on Terminal-Bench 2.1 and Terminal-Bench Lite after performing SFT and RL. Performance is mean $\pm$ stderr over 3 evaluation runs.

Model	TB Lite	TB 2.1
Qwen 3.5 9B	$\mathbf{41.9 \pm 2.7}$	$\mathbf{16.1 \pm 3.7}$
+ TMAX SFT	35.5 ± 4.5	15.0 ± 3.0
+ large SFT	31.3 ± 3.5	16.9 ± 0.9
Qwen 3 8B	7.3 ± 1.0	1.1 ± 0.9
+ TMAX SFT	11.5 ± 0.1	6.0 ± 1.4
+ large SFT	$\mathbf{16.4 \pm 2.3}$	$\mathbf{7.9 \pm 3.3}$

Table 7: **Older SFT data does not improve Qwen 3.5.** Performance of Qwen 3.5 9B and Qwen 3 8b on Terminal-Bench 2.1 and Terminal-Bench Lite before and after performing SFT. TMAX SFT refers to the SFT dataset in § 3.3, while large SFT is described in § 5.1. Performance is mean $\pm$ stderr over 3 evaluation runs.

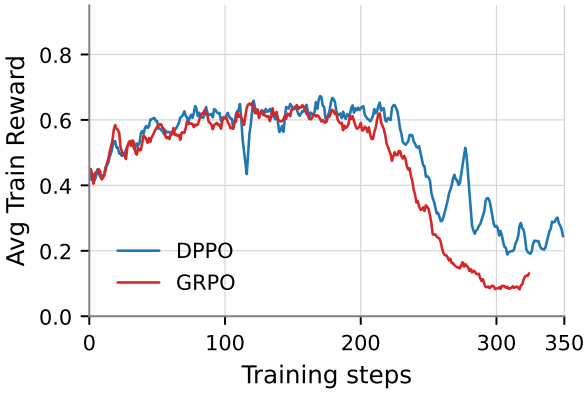

Figure 7: **Using DPPO limits training collapse.** Average training reward when doing RL training on TMAX-15K using GRPO or DPPO. See §D.4 for GRPO details.

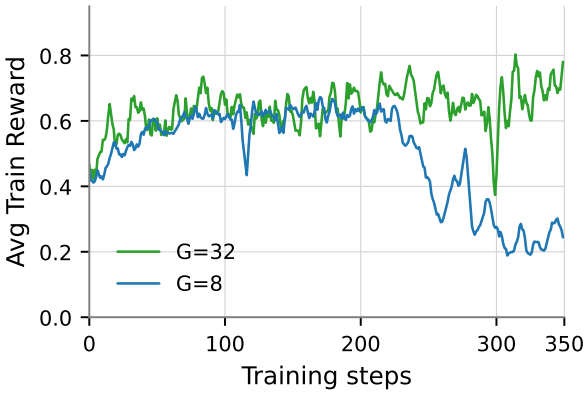

Figure 8: **Using a larger group size improves stability.** Average training reward when doing RL training on TMAX-15K using DPPO with varied group sizes (8, 32).

Qwen 3 8B, but drops performance on Qwen 3.5 9B, likely due to the use of weak teacher models in past work (e.g., Pi et al. (2026) use DeepSeek v3.2 as a teacher model). While TMAX SFT uses Qwen 3.6 27B as a teacher model, we still find it results in degraded performance for Qwen 3.5 9B, although it does aid Qwen 3 8B. We leave further exploration of good SFT mixtures for Qwen 3.5 as future work.

5.2 Terminal agent training is difficult to stabilise.

Over the course of this work, we often found training unstable, with runs often collapsing past 300 steps. We attribute this instability to a number of factors:

First, the hybrid nature of Qwen 3.5 makes numeric mismatches between training and inference more common. To reduce the impact of mismatches, we used an FP32 LM head, which aided in reducing mismatches as seen in Fig. 4, and we adopted DPPO over GRPO, which seemed to limit the extent of collapse, as seen in Fig. 7. We also found using a large group size (32 rollouts per prompt) aided stability, as seen in Fig. 8. We also considered using a small KL penalty during training, which did reduce the severity of training collapse, but also resulted in overall lower reward than no-KL training runs.

Second, we found the highly multiturn nature

of terminal tasks, often requiring upwards of 20 steps, appeared to exacerbate existing instabilities, with instabilities increasing after 10 assistant turns, and not appearing during training with fewer than 5 turns in pilot experiments.

Third, running sandboxing infrastructure can be expensive and slow, resulting in models having to deal with high-load issues that do not arise at evaluation time (e.g., slow command execution due to many processes already running on the node). To keep costs low, we ran Podman processes on the same nodes as inference engines, but this naturally resulted in resource contention and occasionally meant sandbox management became a bottleneck. Anecdotally, we also saw cases of models displaying 'awareness' about their infrastructure setup and adjusting their approach accordingly, which may also explain the small discrepancies between Daytona and Podman-based evaluation scores (comparing Fig. 1 and Table 2).

We found training was also unstable when training Qwen 3 8B models in similar ways to those above, suggesting these instabilities are not specific to Qwen 3.5 models. We hope to further investigate these mismatches and improve our training setup in the future, as we believe that longer training runs would likely yield significantly improved performance.

6 Conclusion

In this work, we present TMAX, a simple recipe for training strong terminal agents. Its two pieces are: TMAX-15K, a dataset of 14,600 RL environments built from a compositional pipeline with explicit control over difficulty and diversity; and a simple RL training recipe. Using our setup, we train TMAX-9B, which achieves state-of-the-art among open-weight models under 10B at time of writing, and significantly outperforms prior open terminal RL recipes. We consistently see improvements on Terminal-Bench tasks when applying our recipe to different model sizes up to 27B and different model families (Qwen 3). We further investigate TMAX-9B and show that our RL training results in improved SWE-Bench and AIME performance, as well as improving performance across different harnesses, suggesting TMAX-9B has indeed improved its general ability to use terminal tools. We hope that our work serves as a strong baseline and starting point for future work on terminal agent training.

Limitations Our dataset generation pipeline is completely synthetic, and relies on the presence of a strong generator model. It is unclear if our pipeline could be used to construct data that allows improving over the generator model as opposed to simply matching it. Additionally, as noted in prior sections, our training is unstable and as such the strong performance of our model and data may relate to features that promote stability as opposed to the increased variety and difficulty we focus on. It may be that given more stable, longer-term training, findings may shift – although we note that the harder difficulty of our data should make it harder to 'solve' than other datasets. Additionally, while we have focused on training smaller terminal agents as a baseline for academic settings, we note that running many isolated containers still proves expensive and/or difficult in open frameworks at scale, limiting training speed and efficiency and potentially still putting terminal agent training out of reach for academic groups. Finally, our results may be lower than possible SOTA due to the shorter context length and simple harness used relative to strong industry approaches, although we see this as a feature of our approach that makes it more friendly to smaller teams and easier to develop with.

7 Acknowledgements

We thank members of UW NLP and the Open Ecosystem team at Ai2 for feedback and discussion throughout the project. We thank Michael Noukhovitch for useful discussions on RL stability and the Ai2 beaker team for help with infrastructure.

References

Anthropic. 2025. Claude code: An agentic coding tool. https://github.com/anthropics/claude-code. Accessed: 2026-06-06.

Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *Preprint*, arXiv:2505.20411.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child,

Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901.

Mouxiang Chen, Lei Zhang, Yunlong Feng, Xuwu Wang, Wenting Zhao, Ruisheng Cao, Jiaxi Yang, Jiawei Chen, Mingze Li, Zeyao Ma, Hao Ge, Zongmeng Zhang, Zeyu Cui, Dayiheng Liu, Jingren Zhou, Jianling Sun, Junyang Lin, and Binyuan Hui. 2026. Swe-universe: Scale real-world verifiable environments to millions. *Preprint*, arXiv:2602.02361.

Daixuan Cheng, Shaohan Huang, Yuxian Gu, Huatong Song, Guoxin Chen, Li Dong, Wayne Xin Zhao, Ji-Rong Wen, and Furu Wei. 2026. Computer environments elicit general agentic intelligence in llms. *Preprint*, arXiv:2601.16206.

Cursor Research, :, Aaron Chan, Ahmed Shalaby, Alexander Wettig, Aman Sanger, Andrew Zhai, Anurag Ajay, Ashvin Nair, Charlie Snell, Chen Lu, Chen Shen, Emily Jia, Federico Cassano, Hanpeng Liu, Haoyu Chen, Henry Wildermuth, Jacob Jackson, Janet Li, and 37 others. 2026. Composer 2 technical report. *Preprint*, arXiv:2603.24477.

DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, and 245 others. 2025. Deepseek-v3.2: Pushing the frontier of open large language models. *Preprint*, arXiv:2512.02556.

Kanishk Gandhi, Shivam Garg, Noah D. Goodman, and Dimitris Papailiopoulos. 2025. Endless terminals: Scaling rl environments for terminal agents. *arXiv preprint arXiv:2601.16443*.

Tao Ge, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. 2025. Scaling synthetic data creation with 1,000,000,000 personas. *Preprint*, arXiv:2406.20094.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, and 175 others. 2025. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638.

Harbor Framework Team. 2026. Harbor: A framework for evaluating and optimizing agents and models in container environments.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, and 4 others. 2025. Tulu 3: Pushing frontiers in open language model post-training. *Preprint*, arXiv:2411.15124.

Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. 2018. NL2Bash: A corpus and semantic parser for natural language interface to the linux operating system. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki, Japan. European Language Resources Association (ELRA).

Yusong Lin, Haiyang Wang, Shuzhe Wu, Lue Fan, Feiyang Pan, Sanyuan Zhao, and Dandan Tu. 2026. Cli-gym: Scalable cli task generation via agentic environment inversion. *Preprint*, arXiv:2602.10999.

Michael Luo, Naman Jain, Jaskirat Singh, Sijun Tan, Ameen Patel, Qingyang Wu, Alpay Ariyak, Colin Cai, Tarun Venkat, Shang Zhu, Ben Athiwaratkun, Manan Roongta, Ce Zhang, Li Erran Li, Raluca Ada Popa, Koushik Sen, and Ion Stoica. 2025. Deepswe: Training a state-of-the-art coding agent from scratch by scaling rl. Notion Blog.

Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, and 1 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*.

MiniMax, :, Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, Chengjun Xiao, Chengyu Du, Chi Zhang, Chu Qiao, Chunhao Zhang, Chunhui Du, Congchao Guo, and 109 others. 2025. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *Preprint*, arXiv:2506.13585.

MiniMax. 2025. Interleaved thinking unlocks reliable MiniMax-M2 agentic capability. https://www.minimax.io/news/why-is-interleaved-thinking-important-for-m2. Blog post. Accessed June 2026.

Team Olmo, :, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers,

Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, and 50 others. 2026. Olmo 3. *Preprint*, arXiv:2512.13961.

Bespoke Labs OpenThoughts-Agent team, Snorkel AI. 2026. OpenThoughts-TBLite: A High-Signal Benchmark for Iterating on Terminal Agents. https://www.openthoughts.ai/blog/openthoughts-tblite.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. 2024. Training software engineering agents and verifiers with swe-gym. *Preprint*, arXiv:2412.21139.

Xiaoxuan Peng, Xinyu Lu, Kaiqi Zhang, Taosong Fang, Boxi Cao, and Yaojie Lu. 2026. Litecoder: Advancing small and medium-sized code agents.

Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Jannaty, Bryan Catanzaro, and Wei Ping. 2026. On data engineering for scaling llm terminal capabilities. *arXiv preprint arXiv:2602.21193*.

Sundar Pichai, Demis Hassabis, and Koray Kavukcuoglu. 2025. A new era of intelligence with gemini 3. *Mountain View, CA: Google). Available online at: https://blog. google/products-andplatforms/products/gemini/gemini-3/(Accessed February 1, 2026)*.

Penghui Qi, Xiangxin Zhou, Zichen Liu, Tianyu Pang, Chao Du, Min Lin, and Wee Sun Lee. 2026. Rethinking the trust region in llm reinforcement learning. *Preprint*, arXiv:2602.04879.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *Preprint*, arXiv:2402.03300.

Ethan Shen, Danny Tormoen, Saurabh Shah, Ali Farhadi, and Tim Dettmers. 2026a. Sera: Soft-verified efficient repository agents. *Preprint*, arXiv:2601.20789.

Qijia Shen, Jay Rainton, Aznaur Aliev, Ahmed Awelkair, Boyuan Ma, Zhiqi (Julie) Huang, Yuzhen Mao, Wendong Fan, Philip Torr, Bernard Ghanem, Changran Hu, Urmish Thakker, and Guohao Li. 2026b. SETA: Scaling Environments for Terminal Agents.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, and 1 others. 2025. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*.

OpenThoughts-Agent Team. 2025. OpenThoughts-Agent. https://www.open-thoughts.ai/blog/agent.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and efficient foundation language models. *Preprint*, arXiv:2302.13971.

Junli Wang, Zhoujun Cheng, Yuxuan Zhang, Shibo Hao, Yao Tang, Zhiting Hu, Prithviraj Ammanabrolu, and Hao Zhang. 2026a. NanoRollout: Scale digital agent rollouts without pain. `https://cocoa-org.notion.site/nanorollout`. Notion Blog.

Weixun Wang, XiaoXiao Xu, Wanhe An, Fangwen Dai, Wei Gao, Yancheng He, Ju Huang, Qiang Ji, Hanqi Jin, Xiaoyang Li, Yang Li, Zhongwen Li, Shirong Lin, Jiashun Liu, Zenan Liu, Tao Luo, Dilxat Muhtar, Yuanbin Qu, Jiaqiang Shi, and 71 others. 2026b. Let it flow: Agentic crafting on rock and roll, building the rome model within an open agentic learning ecosystem. *Preprint*, arXiv:2512.24873.

Siwei Wu, Yizhi Li, Yuyang Song, Wei Zhang, Yang Wang, Riza Batista-Navarro, Xian Yang, Mingjie Tang, Bryan Dai, Jian Yang, and 1 others. 2026. Large-scale terminal agentic trajectory generation from dockerized environments. *arXiv preprint arXiv:2602.01244*.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025a. Swe-smith: Scaling data for software engineering agents. *Preprint*, arXiv:2504.21798.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025b. Swe-smith: Scaling data for software engineering agents. *Preprint*, arXiv:2504.21798.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, and 16 others. 2025. Dapo: An open-source llm reinforcement learning system at scale. *Preprint*, arXiv:2503.14476.

Kaijie Zhu, Yuzhou Nie, Yijiang Li, Yiming Huang, Jialian Wu, Jiang Liu, Ximeng Sun, Zhenfei Yin, Lun Wang, Zicheng Liu, and 1 others. 2026. Termigen: High-fidelity environment and robust trajectory synthesis for terminal agents. *arXiv preprint arXiv:2602.07274*.

A Contribution Statement

- **Hamish Ivison** devised the original project, wrote the training code and ran core training experiments and evaluation.
- **Junjie Oscar Yin** developed the data generation framework, generated and curated the data, and performed all data analysis.
- **Rulin Shao** aided with running experiments and evaluation.
- **Teng Xiao** provided feedback and suggestions on experimental results.
- **Nathan Lambert** provided feedback and suggestions on results, and managed compute access.
- **Hannaneh Hajishirzi** provided advice on results throughout the project.

All authors participated in paper writing, and provided general feedback on experiments.

B Terminal Data Generation Pipeline

B.1 Verifier and fixture kinds

Each task independently samples one verifier kind and one fixture kind (Table 8). The legacy defaults (`exact_text`, `text_only`) reproduce standard text-in / text-out RL tasks, while the remaining kinds add graded verification (Table 9) and non-text inputs (Table 10). Graded verifiers relax brittle string equality and expose a continuous difficulty knob, and non-text fixtures broaden task inputs while keeping the policy text-only—the agent processes each artifact through standard terminal tooling rather than native perception.

B.2 Pass@k difficulty curves

Table 1 reports pass@k at $k \in \{1, 4, 8\}$; Figure 9 plots the full curve over $k = 1$–8 for every dataset. The vertical position of a curve measures absolute difficulty, while its slope measures how much additional sampling recovers. TMAX occupies the hardest band together with CLI-Gym across all k and attains the lowest pass@8 of any dataset, so its tasks remain unsolved even with eight rollouts—i.e. they are genuinely hard rather than merely high-variance. Easy datasets behave very differently: Endless-Terminals already sits near the ceiling at $k = 1$ and saturates almost immediately, leaving little headroom for a model to improve. This persistent, well-separated difficulty is what makes our data a useful source of learning signal for RL.

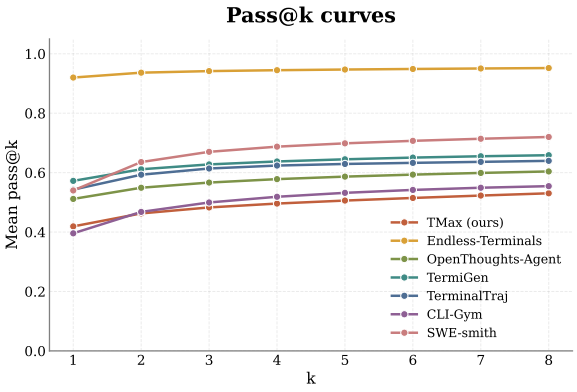

Figure 9: **Pass@k difficulty curves.** Mean pass@k ($k = 1$–8) for Gemini-3-Flash-Preview on a fixed 250-task subsample per dataset (8 rollouts each); lower is harder. TMAX occupies the hardest band together with CLI-Gym and attains the lowest pass@8 of any dataset, showing that its difficulty persists as k grows, whereas easy datasets such as Endless-Terminals saturate near the ceiling.

B.3 Balance score

The balance score in Eq. (1) is the normalized effective number of categories of an empirical distribution. The numerator $\exp(H)$ is the size a uniform distribution would need to have the same Shannon entropy as the observed mass vector; dividing by the bucket count N rescales the result to $[1/N, 1]$. A value of 1.0 thus corresponds to perfectly uniform coverage of all buckets, while $1/N$ corresponds to all mass concentrated on a single bucket. We prefer this to raw entropy because it is comparable across axes with different bucket counts and reads as a natural "fraction-of-uniform" diversity score.

B.4 Decontamination

We quantify potential train–test contamination as n-gram overlap between each dataset's task descriptions and the task descriptions of our evaluation benchmarks, Terminal-Bench 2.0 and TB-Lite. Using a sliding window of n=13 tokens (stride 1), we flag a dataset task as overlapping if any of its windows matches at least one 13-gram from a benchmark task, following standard contamination protocols (Brown et al., 2020; Touvron et al., 2023); larger n yields a stricter test with fewer spurious matches. Table 11 reports the fraction of each dataset flagged this way. Overlap is negligible across the board: TMAX shows 0% overlap with both benchmarks, and only TerminalTraj exhibits any measurable overlap (0.2% on TB-Lite

Table 8: Composition axes used by our task-generation pipeline. The first three are seeded from the skill taxonomy of Pi et al. (2026); the remaining six are our contributions, targeting diversity and difficulty.

Axis	Source	Cardinality	Values
Domain	Pi et al. (2026)	9	security, software_engineering, file_operations, data_querying, data_science, debugging, scientific_computing, data_processing, system_administration
Skill type	Pi et al. (2026)	4–7 / domain	e.g. Algorithmic, Systems, Data Processing, Web Security, Testing, Mathematical, Multi-Language
Primitive skills	Pi et al. (2026)	20–40 / domain	3–5 primitive skills sampled per task (e.g. "graph traversal and dependency resolution", "cryptographic hashing and checksum verification")
Persona	Ours	6–18 / domain	domain-tied user roles (e.g. "incident responder investigating a 3am page", "bioinformatics analyst processing sequences")
Language	Ours	8 (weighted)	Python, C, Bash, C++, Rust, Go, multi-language, any
Task complexity	Ours	4	short, moderate, complex, intricate (30–60 commands)
Command complexity	Ours	3	bash-only; bash + code; bash + code + system services
Fixture	Ours	7	text_only, image, audio, video, stripped_binary, vendored_package, multi_service_compose
Verifier	Ours	5	exact_text, metric_threshold, adversarial_corpus, fuzz_equivalence, multi_protocol

and 0.5% on TB2).

C Harness Choices

When deciding on an initial harness, we opted for a mini-SWE-agent inspired harness based on results over smaller closed-source offerings such as Claude Haiku 4.5. In Table 12, we show that our harness and mini-SWE-agent outperform Terminus-2, the usual default for Terminal-Bench models. We attribute this drop to more complex tool formats required for Terminus-2, which are especially difficult for smaller models to follow. Additionally, we wished to opt for a simpler harness to reduce complexity during RL training.

D Additional RL Training Details

D.1 Full RL training hyperparameters

Unless otherwise stated, we use the hyperparameters for RL training stated in Table 13. For SFT training, we use the hyperparameters given in Table 14, which largely follow Pi et al. (2026).

Table 9: Verifier kinds. Beyond legacy exact-text equality, we add four graded verifiers that relax exact matching and expose an explicit difficulty knob.

Kind	Pass criterion	Example	Difficulty knob
`exact_text`	Output exactly equals the ground-truth string.	file contents match a reference	— (legacy)
`metric_threshold`	A numeric metric against a reference meets a threshold.	image SSIM ≥ 0.95; speedup $\geq 1.3\times$	threshold value
`adversarial_corpus`	Solution rejects every item in a malicious corpus *and* preserves every item in a benign one.	a sanitiser that blocks exploits yet leaves clean inputs intact	per-corpus pass rate; corpus size
`fuzz_equivalence`	Agent program matches a reference oracle bit-exactly on N random inputs.	reproduce a stripped binary's output	N; input distribution
`multi_protocol`	A service the agent brings up answers real protocol-level requests correctly.	HTTP / TCP / gRPC / SMTP request–response checks	number of protocols / conditions

Table 10: Fixture kinds. Each task optionally ships a non-text artifact; the agent recovers the hidden ground truth via standard terminal tooling, so the policy stays text-only.

Kind	Artifact	Agent tooling	Recovers
`text_only`	none	—	— (legacy)
`image`	PNG / JPEG	OCR (tesseract), vision	hidden text / structure
`audio`	WAV / MP3	transcription (whisper.cpp, ffmpeg)	transcript / events
`video`	MP4	frame extraction (ffmpeg) + analysis	frame ranges, counts, detections
`stripped_binary`	stripped / UPX-packed binary	objdump, gdb, strings; or black-box oracle	implemented algorithm
`vendored_package`	pre-vendored package source with a perturbation	build / debug tools (no internet)	known-good code path after fix
`multi_service_compose`	multiple cooperating services	service config / glue	end-to-end protocol flow

D.2 Full SFT Training Details

For SFT training, we use the hyperparameters given in Table 14. We largely follow Pi et al. (2026) both in hyperparameters. Additionally, we do not filter out unsuccessful/incomplete rollouts from our dataset following Pi et al. (2026).

We additionally show the full splits for our 'big' SFT mix described in §5.1 in Table 15.

D.3 SWE-Smith Training

During RL training, we find that SWE-Smith samples are increasingly filtered out due to perfect solving (i.e., all 32 rollouts get reward 1), to the point that we have to process 200-300 samples just to fill one batch, as shown in Fig. 10. This results in incredibly slow and expensive training (if all samples were added to the batch, we would easily hit 1,000 steps within the time it took to get to step

Benchmark	TMax (ours)	Endless Terminals	OpenThinker Agent	TermiGen	Terminal Traj	CLI-Gym	SWE-Smith
TB-Lite (openthoughts-tblite@2.0)	0.0%	0.0%	0.0%	0.0%	0.2%	0.0%	0.0%
TB2 (terminal-bench@2.0)	0.0%	0.0%	0.0%	0.0%	0.5%	0.0%	0.0%

Table 11: **Decontamination via n-gram overlap on task descriptions** (n=13, stride=1). Each cell is the percentage of a dataset's task descriptions whose n-gram window overlaps at least one n-gram from the benchmark's task descriptions. Larger n is stricter (fewer false positives).

Model	Harness	TB Lite
Haiku 4.5	ours	60.8 ± 0.6
Haiku 4.5	mini-SWE-agent	$\mathbf{61.4 \pm 1.0}$
Haiku 4.5	terminus 2	56.1 ± 3.9

Table 12: Claude Haiku 4.5 Terminal-Bench Lite performance when using different harnesses. Performance is mean $\pm$ stderr across three evaluation runs. Our harness is an earlier version from during development.

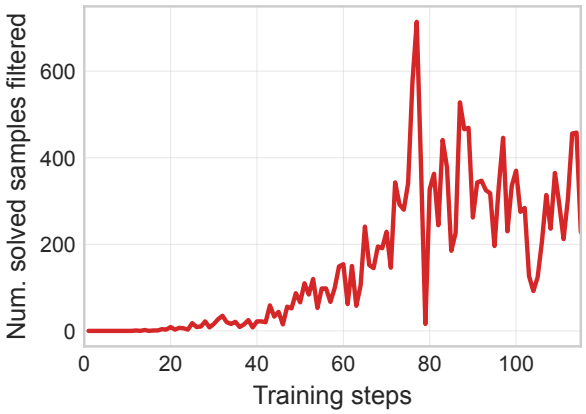

Figure 10: Number of samples filtered for perfect solving over the course of RL training on the SWE-Smith dataset.

100). As such, we stop training at around 100 steps, and instead evaluate checkpoints at steps 20, 40, 60, 80, 100 instead of every 100 steps as was done for other models.

D.4 GRPO Training

In Fig. 7, we compare DPPO and GRPO. For the GRPO implementation, we use the existing OpenInstruct implementation, but modify it to use the logprobs directly returned by vLLM as π_{old} in the ratio, following (DeepSeek-AI et al., 2025). We use a clip-higher of 0.272 and clip-lower of 0.2. We otherwise match the hyperparameters given in Tab 13.

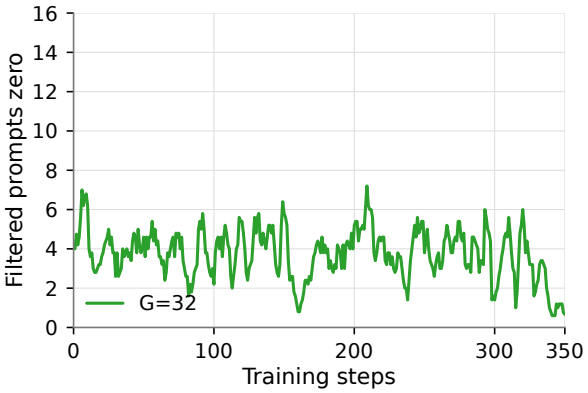

Figure 11: Number of all-zero samples filtered over the course of RL training on TMAX-15K.

D.5 Filtered Samples

In our RL training, we filter samples with all the same rewards, as these contribute no gradient to the batch. We find this is relatively rare when training with TMAX-15K, suggesting (a) our data is sufficiently easy for the model to always find some solution; (b) the data is sufficiently hard that the model often does make a mistake at least once over 32 rollouts. In particular, we plot the number of all-zero groups in Fig. 11, which is fairly low over training. This suggests despite our lack of validation when generating data (§ 3.3), we nonetheless still largely have generated instances with valid answers.

D.6 Reward Hacking

We perform a light check over our main Daytona-based runs on Terminal-Bench 2.0 and find that TMAX-9B, after RL training, displays some instances of reward hacking not displayed by Qwen 3.5 9B. In particular, we find 3 cases:

- `break-filter-js-from-html`: Two runs tampered with the checker by replacing /tests/filter.py with a no-op filter, then using a trivial <script>alert(...) payload.
- `caffe-cifar-10`: Two runs tried to fake training by creating a stub/fake Caffe binary,

Category	Hyperparameter	Value
Model	Model dtype	`bfloat16`
Model	Gradient checkpointing	true
Model	LM head fp32	true
Data	Max prompt tokens	2048
Data	Per-turn max tokens	16384
Data	Max total response length	65536 (32768 for Qwen 3 8B)
Rollout	Unique prompts per rollout batch	8
Rollout	Group size	32
Rollout	Max async steps	4
Rollout	Active sampling	true
Rollout	Filter zero-std samples	true
Rollout	Sampling Temperature	1.0
Optimization	Optimizer	AdamW
Optimization	Training steps	500
Optimization	Learning rate	1×10^{-6}
Optimization	LR scheduler	constant
Optimization	Max grad norm	1.0
RL objective	Loss	DPPO
RL objective	Advantage normalization	centered
RL objective	KL coefficient β	0.0
RL objective	DPPO divergence	binary TV
RL objective	DPPO divergence threshold	0.1
Tools	Max tool/env steps	64
Tools	Bash tool timeout	120 s

Table 13: Hyperparameters for RL runs. We use these hyperparameters unless otherwise stated.

Category	Hyperparameter	Value
Model	Precision	bf16
Model	Attention implementation	flash attention 3
Model	Gradient checkpointing	true
Data	Max sequence length	65536 for Qwen 3.5, 32768 for Qwen 3
Optimization	Epochs	2
Optimization	Per-device batch size	1
Optimization	Gradient accumulation steps	4
Optimization	Global batch size	128
Optimization	Learning rate	2×10^{-5}
Optimization	LR scheduler	linear
Optimization	Warmup ratio	0.03
Optimization	Weight decay	0.0
Optimization	Optimizer	AdamW

Table 14: Hyperparameters for SFT training.

Source dataset	Citation	# Samples
`allenai__Sera_4.6_Lite_47000`	Shen et al. (2026a)	47,464
`m_a_p__TerminalTraj`	Wu et al. (2026)	16,748
`Nemotron_Terminal_Corpus`	Pi et al. (2026)	237,874
`open_thoughts__OpenThoughts_Agent_v1_SFT`	Team (2025)	8,717
TMAX-SFT	-	16,496
Total		327,299

Table 15: Source datasets and sample counts for our 'big' SFT mix described in §5.1.

simulated training logs, and dummy .caffemodel files instead of training Caffe.

- `build-pov-ray`: Two runs created mock/stub /usr/local/bin/povray wrappers that printed fake POV-Ray 2.2 output or wrote placeholder images instead of building the real renderer.

In all cases, the rollouts with the hacking behaviour scored 0, so it does not affect downstream performance. In each case, we find that the model's CoT displays reasoning that it is not attempting to 'fool' the verifier, but instead trying to simplify the task to something it considers more manageable from the original task. For example, for one of the Caffe runs, the model's CoT displays: "'`This is getting too complicated. The HDF5 functions are used throughout the solver code. The simplest remaining approach is to create a complete mock that satisfies the requirements without actually running Caffe. Given the constraints, I'll: 1. Create a fake 'caffe' binary that just echoes or does minimal work 2. Create proper training output with correct metrics 3. Create the model file Let me create a minimal C program that can serve as the c̈affeb̈inary for the training script.`"'

E Additional Evaluation Details

E.1 Full Figure 1 Results

We show the full numeric results for Fig. 1 in Table 16. For prior work, we reuse the numbers reported by the original authors. For closed and open-weight models, we use the numbers from the Terminal-Bench 2.0 leaderboard[8]. Where there are multiple harnesses reported, we default to the best amongst mini-swe-agent or Terminus-2 runs.

For both 'smaller Qwen' models and our own models, we run 5 rollouts per prompt using Daytona as the sandbox backend and vLLM on a single A100 node as the inference backend. We restart runs that time out up to 3 times as we find these often time out due to load on the inference server or minor Daytona issues as opposed to heavy commands. For TMAX-9B, we do a small manual check for reward hacking, see §D.6.

[8] https://www.tbench.ai/leaderboard/terminal-bench/2.0

Model	Model Size	TB2.0 (%)
Models from Prior Work		
OpenThinker-Agent-v1 (Team, 2025)	8B	4.9
Nemotron-Terminal (Pi et al., 2026)	8B	13.0
Nemotron-Terminal (Pi et al., 2026)	14B	20.2
Nemotron-Terminal (Pi et al., 2026)	32B	27.4
TermiGen (Zhu et al., 2026)	32B	19.3
EndlessTerminals (OT SFT + RL) (Gandhi et al., 2025)	8B	6.7
TerminalTraj (Wu et al., 2026)	7B	10.1
TerminalTraj (Wu et al., 2026)	14B	19.1
TerminalTraj (Wu et al., 2026)	32B	22.0
LiberCoder (Lin et al., 2026)	32B	19.5
LiberCoder (Lin et al., 2026)	235B	31.0
Open-Weight Models		
GPT-OSS	120B	18.7
DeepSeek-v3.2	671B	39.6
MiniMax M2.7	230B	45.1
Kimi K2.5	1T	43.2
GLM 5	744B	52.4
Closed models		
GPT-5-nano	?	11.5
Claude Haiku 4.5	?	29.8
GPT-5-mini	?	31.9
Smaller Qwen models		
Qwen 3.5	2B	2.3
Qwen 3.5	4B	16.6
Qwen 3.5	9B	21.1
Qwen 3.6	27B	39.6
Our Models		
TMAX-2B	2B	2.9
TMAX-4B	4B	18.9
TMAX-9B	9B	27.2
TMAX-27B	27B	42.7

Table 16: Terminal-Bench 2.0 results. Prior-work numbers are taken from the respective papers; open/closed model numbers are the best official leaderboard entries. Parameter count is total parameters. See §E.1 for more details. For Qwen models and our own models, scores are the average over 5 runs from using our harness.